

Kali: Agentic Cyber-Defense

Girish Mundra
2024A7PS0993P
BITS Pilani, Pilani Campus

April 30, 2026

Contents

1	Project Overview	2
2	Infrastructure Setup	2
3	Backend Architecture	3
4	System Workflow	3
5	Final API Response Structure	4
6	Key Design Decisions	5
7	Environment Configuration	5
8	Future Improvements	5
9	Conclusion	6

1 Project Overview

This project broadly aims to build an Autonomous AI Interface for Kali Linux - a conversational tool that translates natural language security requests into executable security operations.

The project implements an LLM-driven cybersecurity automation system where natural language inputs are converted into executable security tool operations running inside Dockerized environments.

The system integrates:

- Dockerized Kali Linux execution environment
- Nmap (network scanning)
- Gobuster (directory enumeration)
- OWASP ZAP (web vulnerability scanning)
- Ollama (local)/Gemini LLM (task planning and result analysis)
- FastAPI backend

The final architecture follows a multi-stage agent pipeline:

Natural Language → LLM Planner → Tool Execution → LLM Analyst → Final Report

2 Infrastructure Setup

2.1 Server Environment

- Remote GPU server accessed via SSH
- Docker runtime on host system

2.2 Docker Containers

2.2.1 Kali Linux Container

- Container name: `girish-container`
- Contains: `nmap`, `gobuster`, security utilities
- Used for command execution via Docker SDK

2.2.2 OWASP ZAP Container

- Image: `ghcr.io/zaproxy/zaproxy:stable`
- Runs on port 8080
- Used for spider-based vulnerability scanning

2.2.3 Ollama Container (LLM Fallback/Optional)

- Container name: `ollama-container`
- Runs Ollama framework for local LLM inference
- Model used: Llama3 (CPU-based fallback)
- Purpose:
 1. Backup when Gemini API quota is exhausted
 2. Offline summarization and analysis support
 3. Reduces dependency on external APIs
- Note: GPU acceleration was not available due to missing CUDA support inside container

3 Backend Architecture

The backend is implemented using FastAPI and structured into modular components:

- `main.py` - API entry point and orchestration layer
- `llm.py` - LLM-based planner and summarizer
- `executor.py` - Docker-based command execution
- `validator.py` - security validation layer
- `zap.py` - OWASP ZAP integration

4 System Workflow

Step 1: User Input

User submits a natural language query:

```
scan directory vulnerability scanme.nmap.org
```

Step 2: LLM Planner (Task Generation)

The Gemini LLM converts the input into a structured JSON execution plan:

```
[
  {
    "tool": "nmap",
    "args": "nmap -F scanme.nmap.org"
  },
  {
    "tool": "gobuster",
    "args": "gobuster dir -u http://scanme.nmap.org ..."
  },
  {
    "tool": "zap",
    "args": "scanme.nmap.org"
  }
]
```

This stage replaces traditional rule-based interpretation systems.

Step 3: Validation Layer

Each generated task is validated before execution:

- Only allowed tools: nmap, gobuster, zap
- Blocks unsafe command patterns (rm, shutdown, pipes, etc.)
- Ensures correct JSON schema

Step 4: Tool Execution Layer

Commands are executed inside Dockerized environments:

```
container.exec_run(command)
```

ZAP Execution

OWASP ZAP performs spider-based scanning and extracts:

- Medium and High severity vulnerabilities
- Unique alerts only

Step 5: LLM Analysis Layer

After tool execution, results are passed to Gemini/Ollama for summarization.

The summarizer produces:

- Open port analysis (Nmap)
- Directory findings (Gobuster)
- Security vulnerabilities (ZAP)
- Overall risk assessment
- Actionable recommendations

5 Final API Response Structure

```
{
  "plan": [...],
  "results": {
    "nmap": "...",
    "gobuster": "...",
    "zap": [...]
  },
  "summary": "..."
}
```

6 Key Design Decisions

- LLM is used for BOTH planning and analysis (not execution)
- Tool execution is fully isolated in Docker
- Structured JSON replaces keyword-based parsing
- FastAPI acts as orchestration layer
- Modular design allows easy addition of new tools

7 Environment Configuration

Run backend:

```
GEMINI_API_KEY="your_api_key_here" uvicorn app.main:app --host 0.0.0.0  
--port <port>
```

Eg.

```
GEMINI_API_KEY="my_api_key" uvicorn app.main:app --host 0.0.0.0 --port  
5050
```

8 Future Improvements

The system can be extended in several directions:

1. Multi-Step Adaptive Agent

Enable dynamic chaining of tools based on intermediate results (e.g., Nmap detects open HTTP port → automatically trigger Gobuster/ZAP).

2. Robust JSON Parsing Layer

Implement stricter parsing and validation to handle malformed LLM outputs and ensure deterministic execution.

3. Retry and Fallback Mechanism

Add fallback LLMs (e.g., Ollama) when Gemini quota limits are reached or API failures occur.

4. Structured Tool Argument System

Replace raw command strings with structured argument objects for safer execution.

5. Report Generation Layer

Extend the summarizer into a full report generator (PDF/HTML penetration testing report).

6. Graph-Based Attack Flow Visualization

Represent scan results and tool chains as a directed graph for better analysis of attack surfaces.

9 Conclusion

This project demonstrates a modular LLM-driven cybersecurity automation pipeline combining:

- Natural language understanding via LLM
- Deterministic tool execution in Docker
- Structured validation and orchestration
- AI-based post-execution analysis

The system serves as a foundation for autonomous penetration testing agents and can be extended into multi-step adaptive security frameworks.